Q1: Backend Architecture Overview

> ***Question:*** *Analyze the backend directory structure and give a high-level overview of the architecture. Identify the main entry point, the framework being used, and explain the purpose of the key directories. How is the application initialized?*

Framework

FastAPI (Python 3.13+), running on **Uvicorn** (ASGI). Everything is `async/await` throughout — from DB queries to middleware to WebSocket handlers.

Entry Point — `app/main.py`

This is where the entire application is assembled. It does four things:

1. Lifespan (startup / shutdown)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    start_scheduler()                  # APScheduler cron jobs
    await socket_manager.init_redis()  # Redis Pub/Sub for WebSockets
    yield
    scheduler.shutdown()
    await socket_manager.redis.close()
    await engine.dispose()             # Clean DB connection pool teardown
```

Both the scheduler and Redis are started **non-critically** — if either fails, the API still serves requests normally.

2. Middleware Stack (applied bottom-up by Starlette)

Middleware	Purpose
`CORSMiddleware`	Allows `localhost:5173` , `localhost:3000` , etc.
`SanitizationMiddleware`	XSS prevention on all incoming JSON bodies
`IdempotencyMiddleware`	Deduplicates repeated POST requests via `Idempotency-Key` header

3. All 12 Routers Registered Under `/api/v1/`

4. Global Exception Handlers

- `RequestValidationError` → 422 with structured error detail

- Unhandled `Exception` → 500 with generic message (logged server-side)

Directory Structure & Purpose

```
app/
■■■ main.py              ← Application factory & entry point
■■■ core/                ← Infrastructure: config, DB, security, sockets, rate limiting
■■■ api/
■   ■■■ deps.py          ← Shared FastAPI dependencies (auth, role enforcement)
■   ■■■ v1/              ← All HTTP + WebSocket route handlers
■■■ models/              ← SQLAlchemy ORM table definitions
■■■ schemas/             ← Pydantic request/response models
■■■ services/            ← Business logic (no HTTP awareness)
■■■ middleware/          ← Custom ASGI middleware
```

`core/` — Infrastructure Layer

File	Purpose
`config.py`	Single `Settings` class (pydantic-settings) reads every env var from `.env` . One global `settings` singleton used everywhere.
`database.py`	Creates the async SQLAlchemy engine + session factory. Exports `get_db()` — an async generator injecting a DB session per request.
`security.py`	JWT creation/decoding (python-jose), bcrypt password hashing.
`sockets.py`	`ConnectionManager` — holds active WebSocket connections grouped by role (`ADMIN` / `MANAGER` / `DRIVER`), subscribes to Redis `broadcast` + `alerts` channels, routes messages to the correct role bucket. Has a self-healing supervisor that auto-reconnects if Redis drops.
`rate_limit.py`	Configures `slowapi` limiter instance used on sensitive endpoints.

`api/deps.py` — Dependency Injection (Auth & Authorization)

Three functions form a chain:

```
get_current_user()
  ■■■ get_current_active_user()
```

```
■■■ get_current_user_with_role("ADMIN" | "MANAGER" | "DRIVER")
```

Usage in any router is a single line:

```
user: User = Depends(get_current_user_with_role(UserRole.ADMIN))
```

Pre-built shortcuts: `get_current_admin_user` , `get_current_manager_user` , `get_current_driver_user` .

`api/v1/` — Route Handlers

12 router files, each focused on a role or domain:

File	Prefix	Audience
`auth.py`	`/api/v1/auth`	Everyone — login, refresh, signup, profile, change-password
`admin.py`	`/api/v1/admin`	ADMIN — dashboard stats, rotation creation
`users.py`	`/api/v1/admin/users`	ADMIN — user CRUD
`drivers.py`	`/api/v1/admin/drivers`	ADMIN — driver profiles
`vehicles.py`	`/api/v1/admin/vehicles`	ADMIN — vehicle fleet
`routes.py`	`/api/v1/admin/routes`	ADMIN — bus routes + stops
`tickets.py`	`/api/v1/admin/tickets`	ADMIN — passenger tickets
`manager_ops.py`	`/api/v1/manager`	MANAGER — live fleet, reroute approval
`manager.py`	`/api/v1/manager`	MANAGER — maintenance approvals
`driver.py`	`/api/v1/drivers`	DRIVER — trips, breaks, notifications
`hardware.py`	`/api/v1/hardware`	Hardware devices (API key auth, not JWT)
`websocket.py`	`/ws`	All roles — real-time channel

`models/models.py` — ORM Layer

One file containing all **22 SQLAlchemy table classes** and all Python enums. All models inherit from `Base` (defined in `database.py`).

Key enums: `UserRole` , `DriverStatus` , `VehicleStatus` , `TripStatus` , `ShiftType` , `RotationPosition` .

*Tables are **never created by the app**. Schema is managed exclusively by Alembic (`alembic upgrade head`).*

`schemas/schemas.py` — Validation Layer

One file with all Pydantic models. Each domain follows the pattern:

Schema Suffix	Purpose
`*Base`	Shared fields
`*Create`	POST input — includes validators (e.g. password strength)
`*Update`	PATCH input — all fields optional
`*Response`	Output shape — `from_attributes=True` for ORM serialization

All schemas inherit from `BaseSchema` :

```
class BaseSchema(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

`services/` — Business Logic Layer

Services contain all domain logic and have **no knowledge of HTTP**. They receive DB sessions and domain objects, not `Request` objects. Routers call services; services never call routers.

File	What It Does
`scheduler.py`	APScheduler — 3 cron jobs: daily rotation (5:30 AM), cycling (every 5 min), midnight reset
`rotation_service.py`	Core 3-driver ping-pong algorithm
`break_service.py`	Break validation, fatigue scoring, mandatory break enforcement
`crowding_service.py`	`score = passenger_count / capacity × 100` ; ≥70% alert; ≥90% auto-dispatch
`maintenance_service.py`	Request workflows + IoT threshold → auto EMERGENCY requests
`notification_service.py`	Create/deliver/read notifications + WebSocket broadcast

File	What It Does
`audit_service.py`	Logs every action with user ID, IP, entity, old/new values as JSON
`report_service.py`	Aggregates daily stats, generates PDF (reportlab) and CSV exports

---

`middleware/` — ASGI Middleware

`sanitization.py`

Pure ASGI middleware. Intercepts the raw request body **before** FastAPI reads it, recursively walks every JSON string field and runs `bleach.clean()` to strip HTML/script tags. Only applies to `POST/PUT/PATCH` with `Content-Type: application/json` .

`idempotency.py`

Caches responses keyed by an `Idempotency-Key` header to prevent duplicate operations on retried requests.

Full Request Lifecycle

```
HTTP Request
    ■
    ■■ SanitizationMiddleware   (strips XSS from JSON body)
    ■■ IdempotencyMiddleware    (dedup check)
    ■■ CORSMiddleware           (origin validation)
    ■
    ■■■ Router handler  (api/v1/*.py)
            ■
            ■■ Depends(get_db)            → injects AsyncSession
            ■■ Depends(get_current_user)  → validates JWT, fetches User from DB
            ■■ Depends(role_checker)      → enforces ADMIN / MANAGER / DRIVER
            ■
            ■■■ Service function  (services/*.py)
                    ■
                    ■■ Async DB queries via SQLAlchemy
                    ■■ Audit log write
                    ■■■ ConnectionManager.broadcast_to_role()  (WebSocket push)
```

Generated: 2026-04-16